

Assignment: Reliable UDP Communication in Mininet

NAME: Kaushal kumar ROLL: CSB24069

Objectives:

In this assignment, students will build a UDP client-server application in Mininet and add basic reliability using sequence numbers, ACKs, timeout, and retransmission.

Given Experiment:

Run the final program under the following three Mininet conditions. Run each condition once.

`sudo mn --link tc,bw=5,delay=30ms,loss=0`

`sudo mn --link tc,bw=5,delay=30ms,loss=5`

`sudo mn --link tc,bw=5,delay=30ms,loss=10:`

Short Answer these:

1. Why is ACK needed?
2. Why is timeout needed?
3. Why can duplicate messages occur?
4. What happens when packet loss increases?
5. How is this method similar to TCP?
6. How is it different from TCP?

Experiment 1: Loss = 0%

Here is the output of h1(server) and h2(client):

```
root@Kaushal100:/home/kaushal# cd socket_lab
root@Kaushal100:/home/kaushal/socket_lab# python3 server.py
Server started...
Listening on 10.0.0.1:5000

Received: 1|Message 1 from h2
New message: Message 1 from h2
Sent ACK11

Received: 2|Message 2 from h2
New message: Message 2 from h2
Sent ACK12

Received: 3|Message 3 from h2
New message: Message 3 from h2
Sent ACK13

Received: 4|Message 4 from h2
New message: Message 4 from h2
Sent ACK14

Received: 5|Message 5 from h2
New message: Message 5 from h2
Sent ACK15

Received: 6|Message 6 from h2
New message: Message 6 from h2
Sent ACK16

Received: 7|Message 7 from h2
New message: Message 7 from h2
Sent ACK17

Received: 8|Message 8 from h2
New message: Message 8 from h2
Sent ACK18

Received: 9|Message 9 from h2
New message: Message 9 from h2
Sent ACK19

Received: 10|Message 10 from h2
New message: Message 10 from h2
Sent ACK110

===== SERVER OUTPUT =====
TOTAL_UNIQUE_MESSAGES_RECEIVED=10
TOTAL_DUPLICATES_DETECTED=0
STATUS=SUCCESS
root@Kaushal100:/home/kaushal/socket_lab#
```

```
root@Kaushal100:/home/kaushal# cd socket_lab
root@Kaushal100:/home/kaushal/socket_lab# python3 client.py
Sent: 1|Message 1 from h2
Received: ACK11
Packet 1 successful

Sent: 2|Message 2 from h2
Received: ACK12
Packet 2 successful

Sent: 3|Message 3 from h2
Received: ACK13
Packet 3 successful

Sent: 4|Message 4 from h2
Received: ACK14
Packet 4 successful

Sent: 5|Message 5 from h2
Received: ACK15
Packet 5 successful

Sent: 6|Message 6 from h2
Received: ACK16
Packet 6 successful

Sent: 7|Message 7 from h2
Received: ACK17
Packet 7 successful

Sent: 8|Message 8 from h2
Received: ACK18
Packet 8 successful

Sent: 9|Message 9 from h2
Received: ACK19
Packet 9 successful

Sent: 10|Message 10 from h2
Received: ACK110
Packet 10 successful

===== CLIENT OUTPUT =====
TOTAL_MESSAGES=10
LOSS_PERCENT=0%
TIMEOUT=1.5
TOTAL_PACKETS_SENT=10
TOTAL_RETRANSMISSIONS=0
TRANSFER_TIME_SECONDS=1.3811
STATUS=SUCCESS
root@Kaushal100:/home/kaushal/socket_lab#
```

Experiment 2: Loss = 5%

Here is the image of h1 and h2:

```
root@Kaushal100:/home/kaushal/socket_lab# python3 server.py
Server started...
Listening on 10.0.0.1:5000

Received: 1|Message 1 from h2
New message: Message 1 from h2
Sent ACK11

Received: 2|Message 2 from h2
New message: Message 2 from h2
Sent ACK12

Received: 3|Message 3 from h2
New message: Message 3 from h2
Sent ACK13

Received: 4|Message 4 from h2
New message: Message 4 from h2
Sent ACK14

Received: 5|Message 5 from h2
New message: Message 5 from h2
Sent ACK15

Received: 6|Message 6 from h2
New message: Message 6 from h2
Sent ACK16

Received: 7|Message 7 from h2
New message: Message 7 from h2
Sent ACK17

Received: 7|Message 7 from h2
Duplicate message detected: SEQ 7
Sent ACK17

Received: 8|Message 8 from h2
New message: Message 8 from h2
Sent ACK18

Received: 9|Message 9 from h2
New message: Message 9 from h2
Sent ACK19

Received: 10|Message 10 from h2
New message: Message 10 from h2
Sent ACK110

===== SERVER OUTPUT =====
TOTAL_UNIQUE_MESSAGES_RECEIVED=10
TOTAL_DUPLICATES_DETECTED=1
STATUS=SUCCESS
root@Kaushal100:/home/kaushal/socket_lab#
```

```
root@Kaushal100:/home/kaushal/socket_lab# python3 client.py
Sent: 1|Message 1 from h2
Received: ACK11
Packet 1 successful

Sent: 2|Message 2 from h2
Timeout for SEQ 2. Retransmitting...

Sent: 2|Message 2 from h2
Received: ACK12
Packet 2 successful

Sent: 3|Message 3 from h2
Received: ACK13
Packet 3 successful

Sent: 4|Message 4 from h2
Received: ACK14
Packet 4 successful

Sent: 5|Message 5 from h2
Received: ACK15
Packet 5 successful

Sent: 6|Message 6 from h2
Timeout for SEQ 6. Retransmitting...

Sent: 6|Message 6 from h2
Received: ACK16
Packet 6 successful

Sent: 7|Message 7 from h2
Received: ACK17
Packet 7 successful

Sent: 8|Message 8 from h2
Timeout for SEQ 8. Retransmitting...

Sent: 8|Message 8 from h2
Received: ACK18
Packet 8 successful

Sent: 9|Message 9 from h2
Received: ACK19
Packet 9 successful

Sent: 10|Message 10 from h2
Received: ACK110
Packet 10 successful

===== CLIENT OUTPUT =====
TOTAL_MESSAGES=10
LOSS_PERCENT=5%
TIMEOUT=1.5
TOTAL_PACKETS_SENT=13
TOTAL_RETRANSMISSIONS=3
TRANSFER_TIME_SECONDS=5.8986
STATUS=SUCCESS
```

Experiment 3 :Loss=10%

Here is the screenshot of H1 and H2

```
root@Kaushal100:/home/kaushal# cd socket_lab
root@Kaushal100:/home/kaushal/socket_lab# python3 server.py
Server started...
Listening on 10.0.0.1:5000

Received: 1|Message 1 from h2
New message: Message 1 from h2
Sent ACK11

Received: 2|Message 2 from h2
New message: Message 2 from h2
Sent ACK12

Received: 3|Message 3 from h2
New message: Message 3 from h2
Sent ACK13

Received: 4|Message 4 from h2
New message: Message 4 from h2
Sent ACK14

Received: 5|Message 5 from h2
New message: Message 5 from h2
Sent ACK15

Received: 6|Message 6 from h2
New message: Message 6 from h2
Sent ACK16

Received: 7|Message 7 from h2
New message: Message 7 from h2
Sent ACK17

Received: 8|Message 8 from h2
New message: Message 8 from h2
Sent ACK18

Received: 8|Message 8 from h2
Duplicate message detected: SEQ 8
Sent ACK18

Received: 9|Message 9 from h2
New message: Message 9 from h2
Sent ACK19

Received: 10|Message 10 from h2
New message: Message 10 from h2
Sent ACK110

===== SERVER OUTPUT =====
TOTAL_UNIQUE_MESSAGES_RECEIVED=10
TOTAL_DUPLICATES_DETECTED=1
STATUS=SUCCESS
root@Kaushal100:/home/kaushal/socket_lab#
```

H2:

```
Received: ACK11
Packet 1 successful

Sent: 2|Message 2 from h2
Received: ACK12
Packet 2 successful

Sent: 3|Message 3 from h2
Received: ACK13
Packet 3 successful

Sent: 4|Message 4 from h2
Received: ACK14
Packet 4 successful

Sent: 5|Message 5 from h2
Received: ACK15
Packet 5 successful

Sent: 6|Message 6 from h2
Timeout for SEQ 6. Retransmitting...

Sent: 6|Message 6 from h2
Received: ACK16
Packet 6 successful

Sent: 7|Message 7 from h2
Received: ACK17
Packet 7 successful

Sent: 8|Message 8 from h2
Timeout for SEQ 8. Retransmitting...

Sent: 8|Message 8 from h2
Received: ACK18
Packet 8 successful

Sent: 9|Message 9 from h2
Received: ACK19
Packet 9 successful

Sent: 10|Message 10 from h2
Received: ACK110
Packet 10 successful

===== CLIENT OUTPUT =====
TOTAL_MESSAGES=10
LOSS_PERCENT=10
TIMEOUT=1.5
TOTAL_PACKETS_SENT=12
TOTAL_RETRANSMISSIONS=2
TRANSFER_TIME_SECONDS=4.4571
STATUS=SUCCESS
root@Kaushal100:/home/kaushal/socket_lab#
```

Q1. Why is ACK needed?

Ans: ACK (Acknowledgement) is needed to confirm that the receiver has successfully received a packet. In UDP, packets can be lost because it does not provide built-in reliability. The sender waits for an ACK after sending each packet. If the ACK is not received within the timeout period, the sender assumes the packet or ACK was lost and retransmits the packet. Therefore, ACK helps achieve reliable communication in the Stop-and-Wait protocol.

Q2. Why is timeout needed?

Timeout is needed because the sender cannot wait forever for an ACK from the receiver. If a packet or its ACK is lost, the timeout allows the sender to detect the loss and retransmit the same packet. This helps ensure that all messages are delivered successfully. In the Stop-and-Wait protocol, timeout plays an important role in providing reliability over UDP.

Q3. Why can duplicate messages occur?

Duplicate messages can occur when the sender does not receive an ACK within the timeout period and retransmits the same packet. Sometimes the original packet was received successfully, but the ACK sent by the receiver was lost. The receiver then receives the same packet again and identifies it as a duplicate using the sequence number. This is a common situation in Stop-and-Wait reliable communication over UDP.

Q4. What happens when packet loss increases?

When packet loss increases, more data packets or ACK packets may be lost during transmission. This causes the sender to wait for the timeout and retransmit the lost packets. As a result, the number of retransmissions and the total transfer time

increase. Higher packet loss reduces communication efficiency and may lead to more duplicate packets at the receiver.

Q5. How is this method similar to TCP?

This Stop-and-Wait method is similar to TCP because both provide reliable communication by using acknowledgements (ACKs), sequence numbers, and retransmissions. They ensure that lost packets are sent again and that data reaches the receiver correctly. Both methods also use mechanisms to detect duplicate packets and maintain reliable data transfer over an unreliable network.

Q6. How is it different from TCP?

This method is different from TCP because it implements only basic reliability using sequence numbers, ACKs, timeouts, and retransmissions. TCP provides additional features such as connection establishment, flow control, congestion control, and automatic error handling. TCP also manages data transfer efficiently, whereas this Stop-and-Wait method sends only one packet at a time and is slower. Therefore, it